Iterator

Iterator · Round-robin assign jobs to workers · Build with Waqar

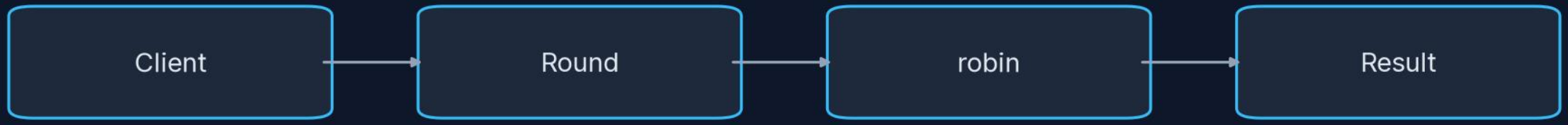
Today's Hook

- Today's force: Round-robin assign jobs to workers
- We earn Iterator by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: RoundRobin

The Problem

- Scenario: Round-robin assign jobs to workers
- Constraints: solve «Round-robin assign jobs to workers» with Iterator — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Round-robin assign jobs to workers
- Success looks like: w1 w2 w1

Structure



Iterator: Round-robin assign jobs to workers

Real-World Scenario

- Running example: Round-robin assign jobs to workers
- Key types: RoundRobin
- Happy-path stdout cue: w1 w2 w1
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator.problem;

import java.util.*;

public final class NaiveAssign {
  private int i; private final List<String> w;
  public NaiveAssign(List<String> w){ this.w=w; }
  public String next(){ return w.get(i++ % w.size()); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Round-robin assign jobs to workers
- Without Iterator, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

final class RoundRobin implements Iterable<String> {
  private final List<String> workers; private int i;
  RoundRobin(List<String> workers){ this.workers=List.copyOf(workers); }
  public Iterator<String> iterator(){
    return new Iterator<>(){
      public boolean hasNext(){ return !workers.isEmpty(); }
      public String next(){ if(workers.isEmpty()) throw new NoSuchElementException();
        String w=workers.get(i % workers.size()); i++; return w; }
    };
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.iterator.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Iterator: round-robin workers ===");
    System.out.println("--- naive ---");
var n=new NaiveAssign(List.of("w1","w2")); System.out.println(n.next()+" "+n.next()+" "+n.next());
System.out.println("--- iterator ---");
var it=new RoundRobin(List.of("w1","w2")).iterator();
System.out.println(it.next()+" "+it.next()+" "+it.next());
  }
}
```

Walkthrough Output

```
=== Iterator: round-robin workers ===
--- naive ---
w1 w2 w1
--- iterator ---
w1 w2 w1
```

Step-by-Step Trace

- Match stdout to stages — look for: w1 w2 w1
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 18-iterator ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Iterator
// Prefer records + sealed where they shrink the demo.
// Problem under test: Round-robin assign jobs to workers
```

Trade-offs

- Use Iterator when «Round-robin assign jobs to workers» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Iterator in product language.

Related Patterns

- Name the pattern usually confused with Iterator and separate the forces.
- Sketch the roles for «Round-robin assign jobs to workers» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Round-robin assign jobs to workers
- Naive pain → Iterator boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.